



Dubbo介绍

杨壮

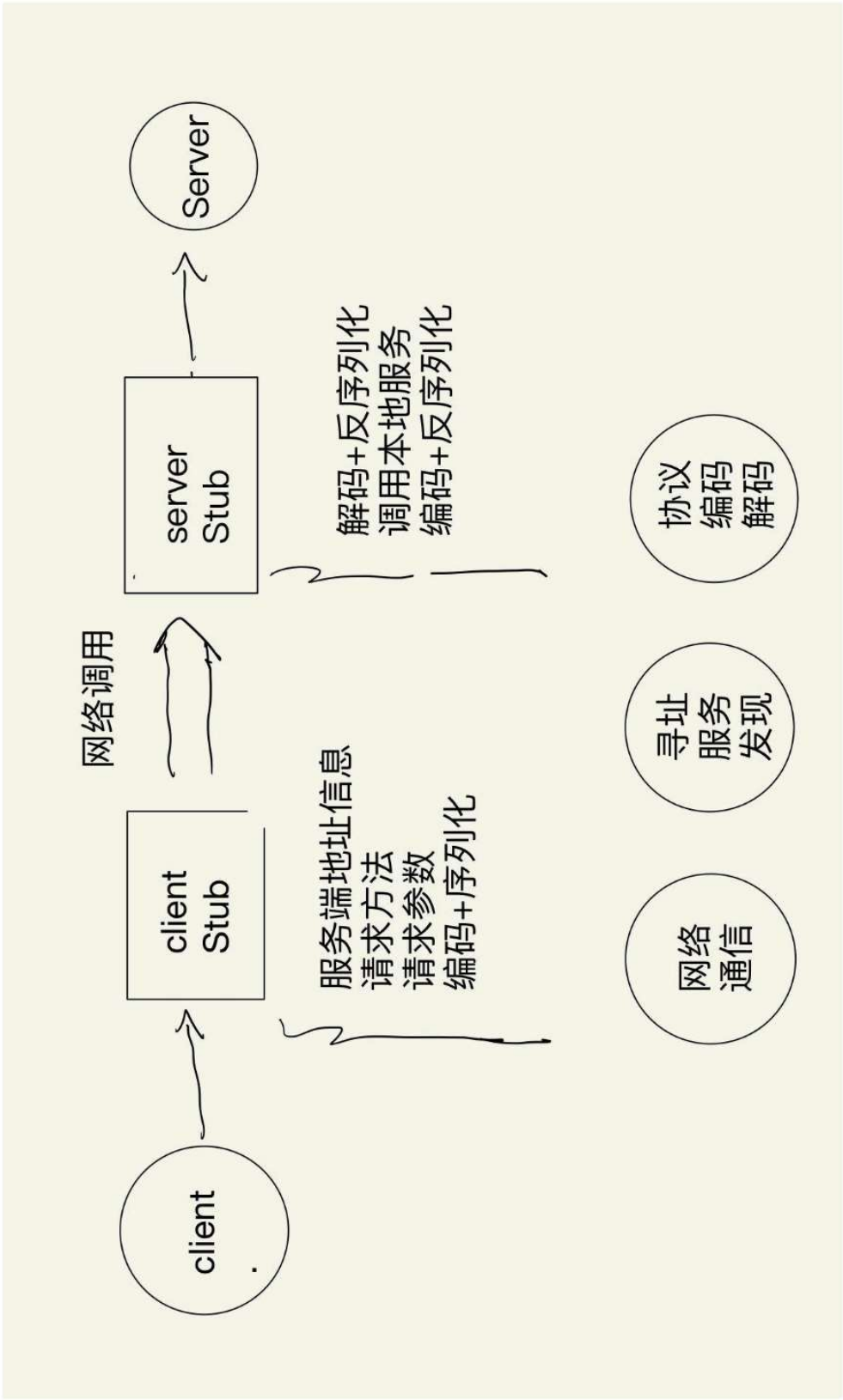
Dubbo是什么

Apache Dubbo 是一款 RPC 服务开发框架，用于解决微服务架构下的服务治理与通信问题。

使用 Dubbo 开发的微服务原生具备相互之间的远程地址发现与通信能力，利用 Dubbo 提供的丰富服务治理特性，可以实现诸如服务发现、负载均衡、流量调度等服务治理诉求。

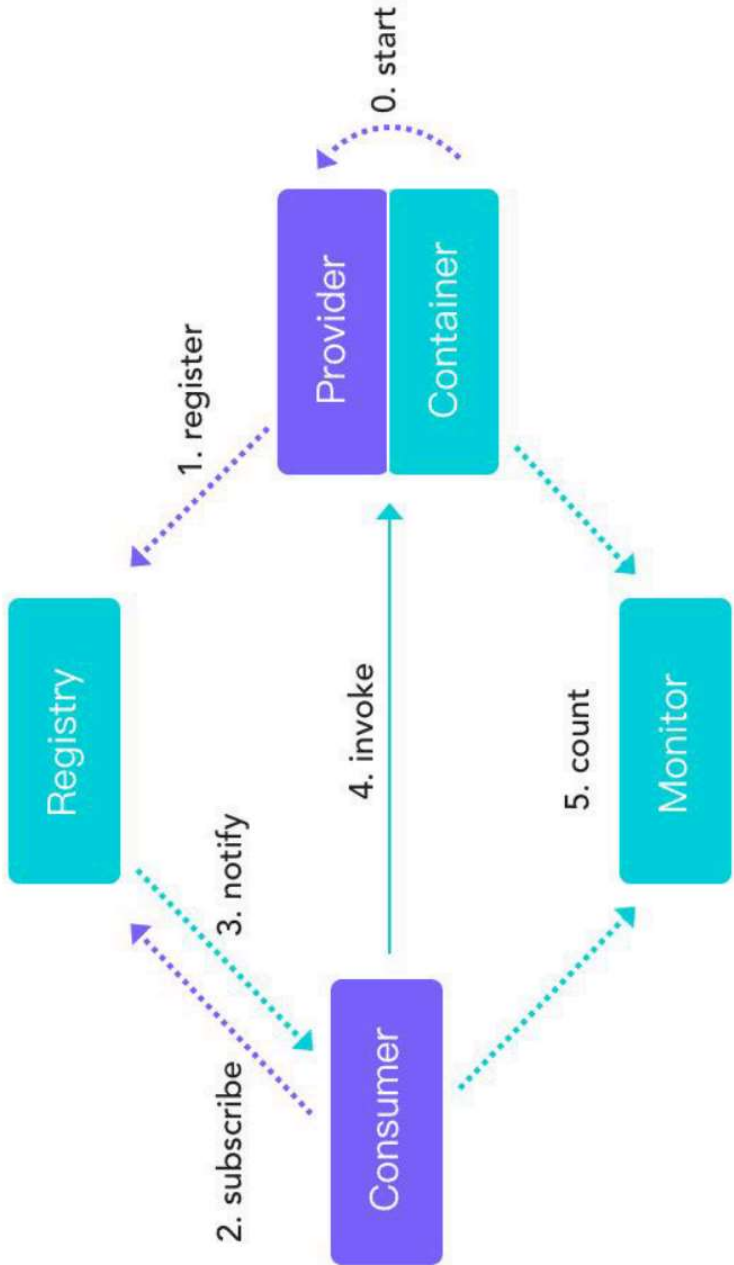


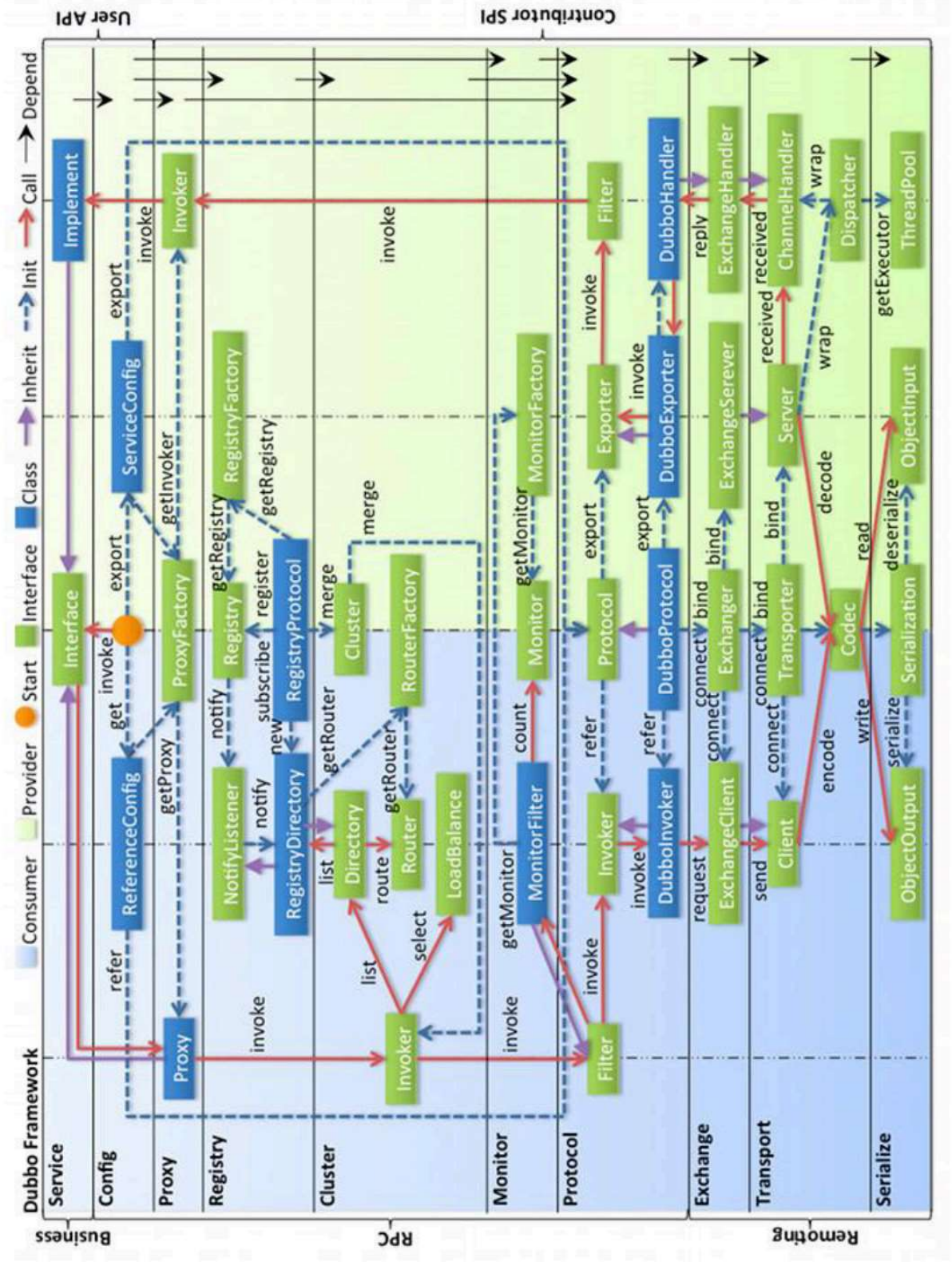
RPC需要什么？



Dubbo Architecture

.....▶ init ▶ async —▶ sync





Part2 Dubbo框架的基石

了解dubbo的第一步:

了解SPI和URL机制

dubbo通过SPI和URL实现了组件的高度可定制化:
在dubbo中几乎所有组件都可以被定制:

比如:

注册中心 负载均衡 序列化协议 传输层 dubbo线程池等等

了解SPI和URL机制

SPI机制主要是提供扩展点的实现

URL则可以配置选择哪个扩展点，并且采用哪些配置



了解SPI和URL机制

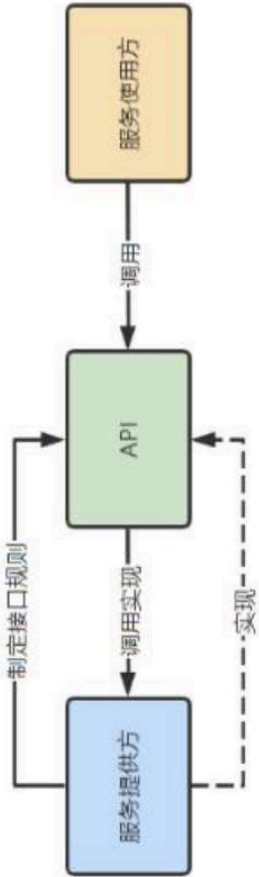
```

/**
 * Init LoadBalance.
 * <p>
 * if invokers is not empty, init from the first invoke's url and invocation
 * if invokes is empty, init a default LoadBalance(RandomLoadBalance)
 * </p>
 *
 * @param invokers invokers
 * @param invocation invocation
 * @return LoadBalance instance. if not need init, return null.
 */
protected LoadBalance initLoadBalance(List<Invoker<T>> invokers, Invocation invocation) {
    if (CollectionUtils.isEmpty(invokers)) {
        return ExtensionLoader.getExtensionLoader(LoadBalance.class).getExtension(invokers.get(0).getUrl()
            .getMethodParameter(RpcUtils.getMethodName(invocation), LOADBALANCE_KEY, DEFAULT_LOADBALANCE));
    } else {
        return ExtensionLoader.getExtensionLoader(LoadBalance.class).getExtension(DEFAULT_LOADBALANCE);
    }
}

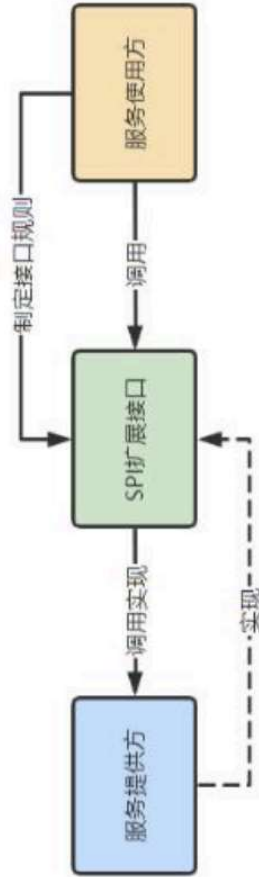
```

如何做到微内核插件化:

API



SPI



如何做到微内核插件化:

SPI	Java SPI方案: 通过ServiceLoader加载Interface所有实现
Dubbo SPI方案	性能考虑 IOC依赖注入实现 自适应 (加入Url参数判断, 动态决定) 根据配置选择性激活
@SPI	@Adaptive @Activate @Inject

如何做到微内核插件化:

```
*/
@SPI(RandomLoadBalance.NAME)
public interface LoadBalance {

    /**
     * select one invoker in List.
     *
     * @param invokers invokers.
     * @param url refer url
     * @param invocation invocation.
     * @return selected invoker.
     */
    @Adaptive("loadbalance")
    <T> Invoker<T> select(List<Invoker<T>> invokers, URL url, Invocation invocation) throws RpcException;
```

如何做到微内核插件化:

```
private Class<?> createAdaptiveExtensionClass() {  
    String code = new AdaptiveClassCodeGenerator(type, cachedDefaultName).generate();  
    ClassLoader classLoader = findClassLoader();  
    org.apache.dubbo.common.compiler.Compiler compiler =  
        ExtensionLoader.getExtensionLoader(org.apache.dubbo.common.compiler.Compiler.class).getAdaptiveExtension();  
    return compiler.compile(code, classLoader);  
}
```

```
/**  
 * Code generator for Adaptive class  
 */
```

```
public class AdaptiveClassCodeGenerator {
```

手工拼装代码实现

```

    */
    @Documented
    @Retention(RetentionPolicy.RUNTIME)
    @Target({ElementType.TYPE, ElementType.METHOD})
    public @interface Activate {
        /** Activate the current extension when one of the groups matches. The group passed into ...*/
        String[] group() default {};

        /** Activate the current extension when the specified keys appear in the URL's parameters. ...*/
        String[] value() default {};

        /** Relative ordering info, optional ...*/
        @Deprecated
        String[] before() default {};

        /** Relative ordering info, optional ...*/
        @Deprecated
        String[] after() default {};

        /** Absolute ordering info, optional ...*/
        int order() default 0;
    }

```



```

    */
    @Activate(group = CONSUMER, value = ACTIVES_KEY)
    public class ActiveLimitFilter implements Filter, Filter.Listener {

```

```

        String[] activateGroup, activateValue;

        if (activate instanceof Activate) {
            activateGroup = ((Activate) activate).group();
            activateValue = ((Activate) activate).value();
        } else {...}

        if (isMatchGroup(group, activateGroup)
            && !names.contains(name)
            && !names.contains(REMOVE_VALUE_PREFIX + name)
            && isActive(activateValue, url)
            && !loadedNames.contains(name)) {
            activateExtensionsMap.put(getExtensionClass(name), getExtension(name));
            loadedNames.add(name);
        }
    }
}

```

如何做到微内核插件化：

@SPI 标注插入点，并提供默认

@Adaptive 重新生成代码，加入Url参数去做自适应判断

@Activate 通过group, value, order去对激活进行配置

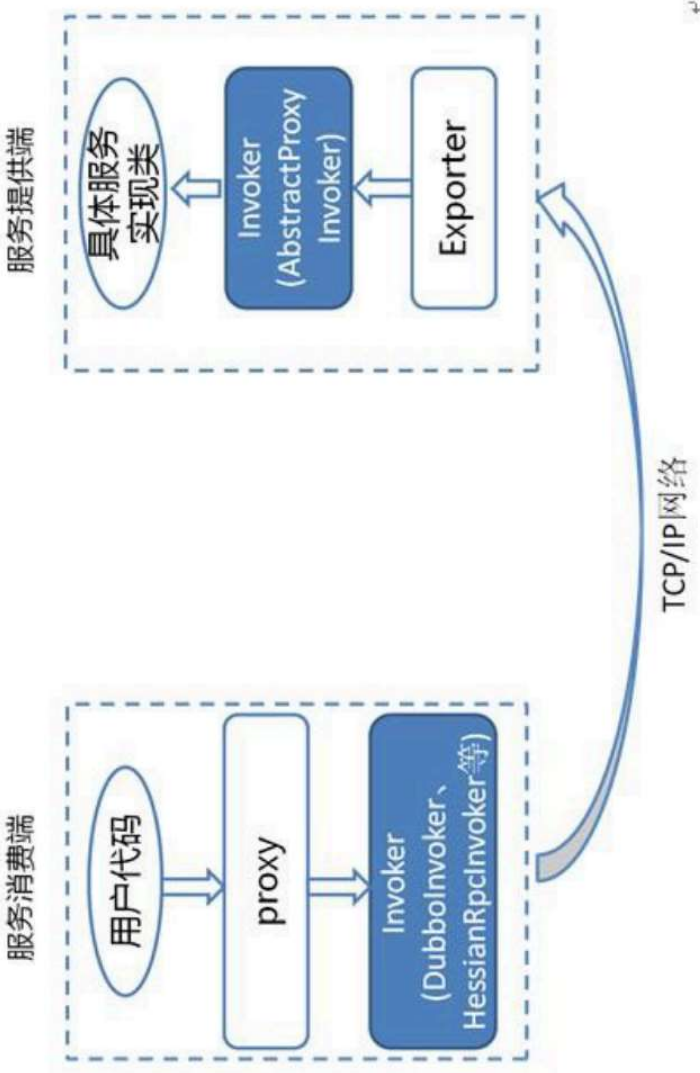
如何通过SPI注入一个自己的实现：

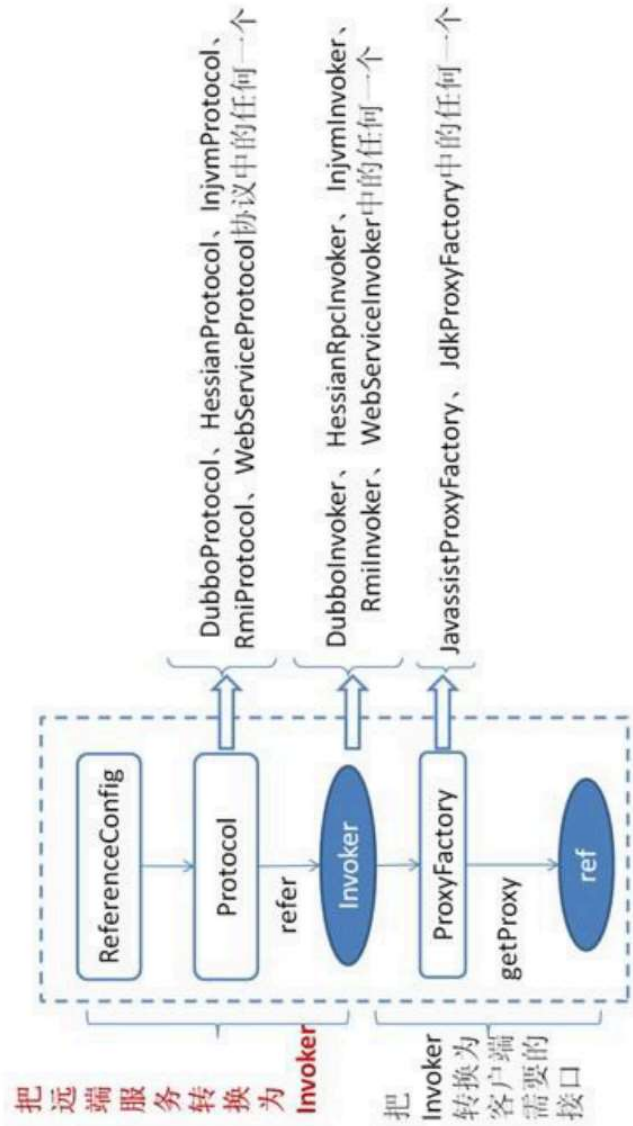
resources/META-INF/dubbo

```
proxy=com.ctrip.flight.intl.engine.coordinator.loadbalance.LoadBalanceProxy  
shortestRTLastNTimes=com.ctrip.flight.intl.engine.coordinator.loadbalance.ShortestRTLastNTimesLoadBalance  
adaptive=com.ctrip.flight.intl.engine.coordinator.loadbalance.AdaptiveLoadBalance
```

修改URL

Part3 Invoke流程





客户端服务调用Invoke流程:

```
public interface SalesControlService {  
    /**  
     * query SalesControlService  
     * @param request  
     * @return response  
     */  
}
```

```
SalesControlResponse query(SalesControlRequest request);
```

```
CompletableFuture<SalesControlResponse> queryAsync(SalesControlRequest request);
```

```
CompletableFuture<SalesControlResponse> future = salesControlService.queryAsync(request);
```

SalesControlService是一个dubbo的client，通过调用queryAsync方法发起一次RPC调用，获取服务端结果

客户端服务调用Invoke流程:

集群容错选择

```
// The invoker wrap sequence would be: ZoneAwareClusterInvoker(StaticDirectory) -> FailOverClusterInvoker  
invoker = Cluster.getCluster(cluster, wrap: false).join(new StaticDirectory(registryURL, invokers));
```

Cluster扩展点选择:
默认是failOverCluster



```
@SPI(Cluster.DEFAULT)  
public interface Cluster {  
    String DEFAULT = "failover";  
}
```



```

//
public class FailoverClusterInvoker<T> extends AbstractClusterInvoker<T> {

```

```

// retry loop.
RpcException le = null; // Last exception.
List<Invoker<T>> invoked = new ArrayList<~>(copyInvokers.size()); // invoked invokers.
Set<String> providers = new HashSet<String>(len);
for (int i = 0; i < len; i++) {
    //Reselect before retry to avoid a change of candidate `invokers`.
    //NOTE: if `invokers` changed, then `invoked` also lose accuracy.

```

```

    if (i > 0) {...}

```

```

    Invoker<T> invoker = select(loadbalance, invocation, copyInvokers, invoked);
    invoked.add(invoker);

```

```

    RpcContext.getContext().setInvokers((List) invoked);

```

```

    try {

```

```

        Result result = invoker.invoke(invocation);

```

```

        if (le != null && Logger.isWarnEnabled()) {

```

```

methods=queryAccountPricePolicy
pid=36
qloglevel=4
release=2.7.7-ctrip.13
retries=0
revision=1.1.41
serviceId=flight.computeengine.salesco

```

客户端服务调用Invoke流程：

集群容错选择

Failfast Cluster

快速失败，只发起一次调用，失败立即报错。通常用于非幂等性的写操作，比如新增记录。

Failsafe Cluster

失败安全，出现异常时，直接忽略。通常用于写入审计日志等操作。

Failback Cluster

失败自动恢复，后台记录失败请求，定时重发。通常用于消息通知操作。

Forking Cluster

并行调用多个服务器，只要一个成功即返回。通常用于实时性要求较高的读操作，但需要浪费更多服务资源。可通过 `forks="2"` 来设置最大并行数。

Broadcast Cluster

广播调用所有提供者，逐个调用，任意一台报错则报错。通常用于通知所有提供者更新缓存或日志等本地资源信息。

客户端服务调用Invoke流程:

负载均衡选择:

```
LoadBalance loadbalance = initLoadBalance(invokers, invocation);

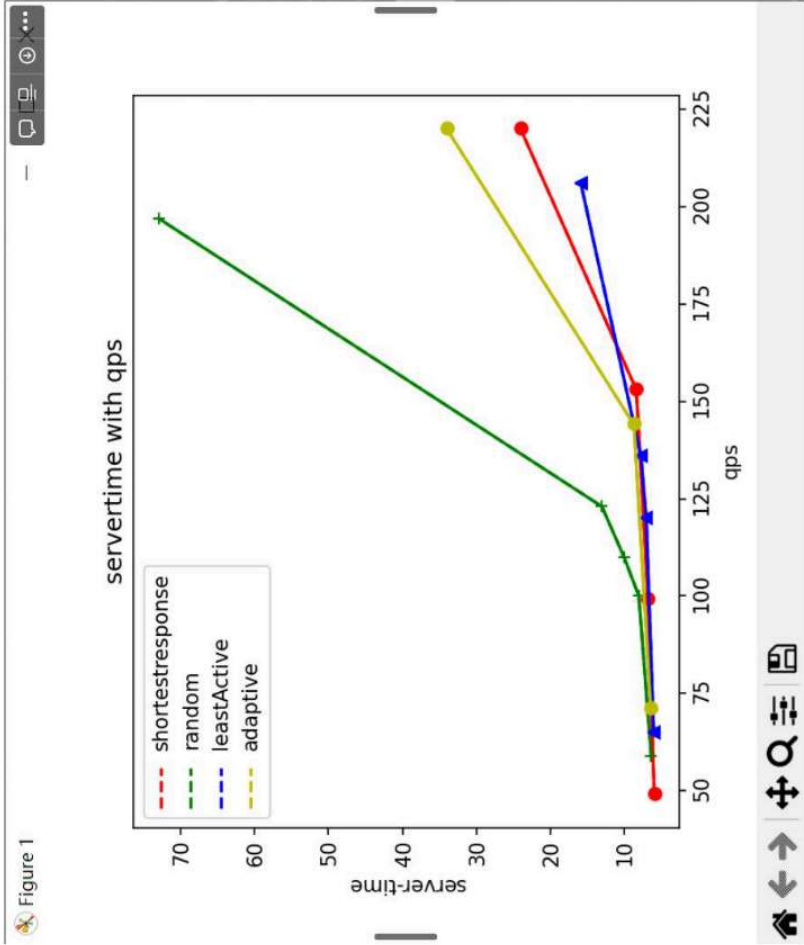
/**
 * @SPI(RandomLoadBalance.NAME)
 * public interface LoadBalance {
 *
 * /**
 *  * select one invoker in list.
 *  *
 *  * @param invokers    invokers.
 *  * @param url         refer url
 *  * @param invocation invocation.
 *  * @return selected invoker.
 *  */
 *
 * @Adaptive("loadbalance")
 * <T> Invoker<T> select(List<Invoker<T>> invokers, URL url, Invocation invocation) throws RpcException;
```

客户端服务调用Invoke流程:

负载均衡算法:

算法	特性	备注
RandomLoadBalance	加权随机	默认算法，默认权重相同
RoundRobinLoadBalance	加权轮询	借鉴于 Nginx 的平滑加权轮询算法，默认权重相同，
LeastActiveLoadBalance	最少活跃优先 + 加权随机	背后是能者多劳的思想
ShortestResponseLoadBalance	最短响应优先 + 加权随机	更加关注响应速度
ConsistentHashLoadBalance	一致性 Hash	确定的入参，确定的提供者，适用于有状态请求

客户端服务调用Invoke流程:



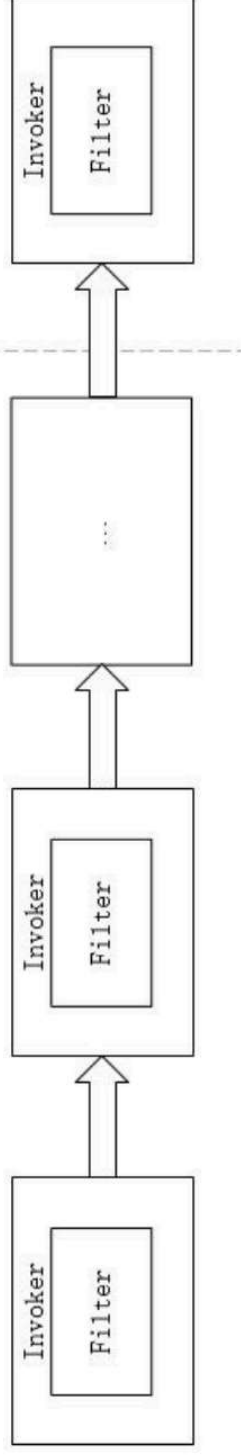
CPU密集型应用可以选择LeastActive算法尝试下
可以提升集群机器的饱和点

```
loadbalance="leastactive"  actives="300"
```

actives需要配置，才能激活Filter

客户端服务调用Invoke流程:

filter责任链: client Filter, 根据group筛选, 并且会按照value判断是否激活, 最后以order排序并组成责任链



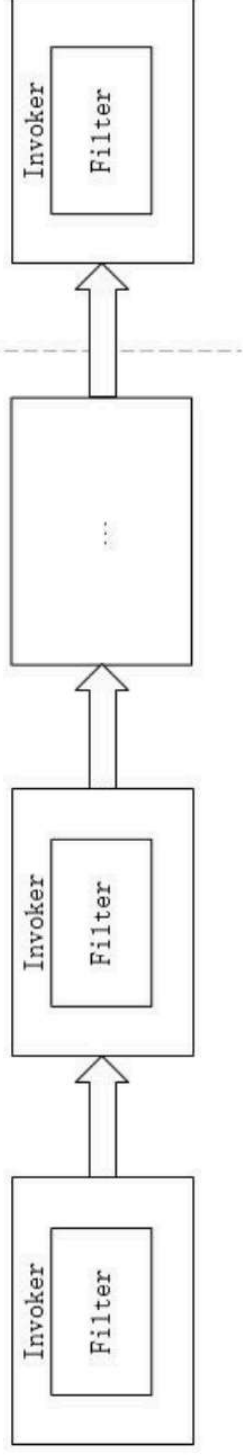
```
*/
```

```
@Activate(group = CommonConstants.CONSUMER, value = CDubboConstants.CDUBBO_ENABLED, order = 290000)  
public class TraceDubboClientFilter implements Filter {
```

```
    ...
```


客户端服务调用Invoke流程:

filter责任链: client Filter, 根据group筛选, 并且会按照value判断是否激活, 最后以order排序并组成责任链

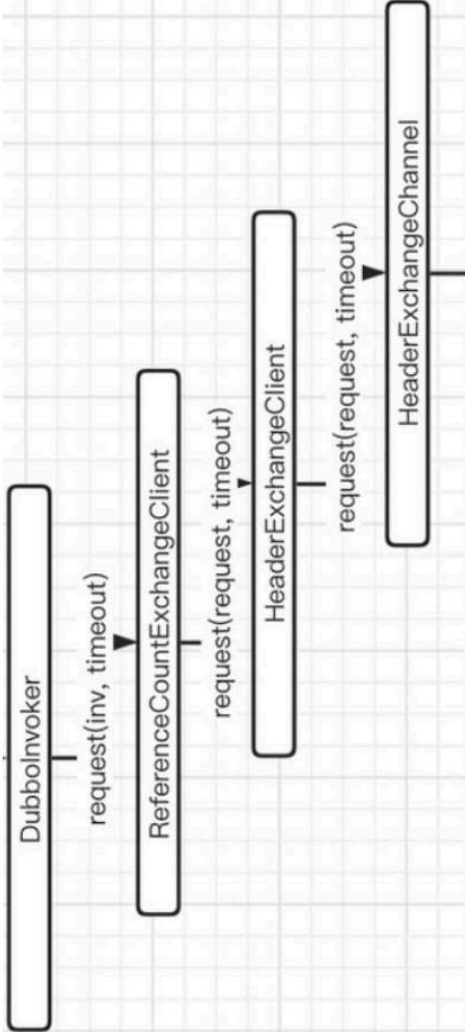


```
*/
```

```
@Activate(group = CommonConstants.CONSUMER, value = CDubboConstants.CDUBBO_ENABLED, order = 290000)  
public class TraceDubboClientFilter implements Filter {
```

```
    ...
```

客户端服务调用Invoke流程:



这个主要是将invocation封装成Request对象，然后将channel，request，timeout 绑到一个Future里面，将Future对象返回

客户端服务调用Invoke流程:

```
// create request.  
Request req = new Request();  
req.setVersion(Version.getProtocolVersion());  
req.setTwoWay(true);  
req.setData(request);  
DefaultFuture future = DefaultFuture.newFuture(channel, req, timeout, executor);
```

这里封装的Future也是dubbo异步调用的实现，会在后面展开

客户端服务调用Invoke流程:

后续Sent的时候会走到Netty的Pipeline，
由于是请求发送，走的是逆序流程

```
pipeline.addLast( name: "decoder", adapter.getDecoder());  
pipeline.addLast( name: "encoder", adapter.getEncoder());  
pipeline.addLast( name: "handler", nettyHandler);
```

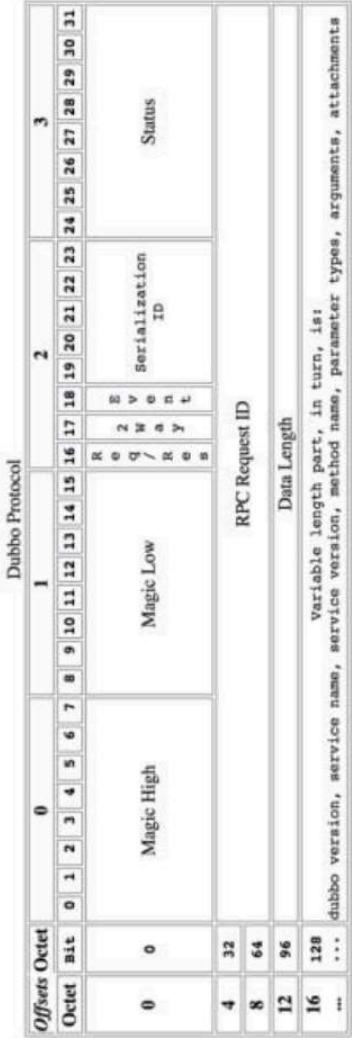
NettyHandler里面会调用到encoder，encoder会调用Codec
Codec里面封装了序列化和反序列化方法。

客户端服务调用Invoke流程:

```
}  
protected void encodeRequest(Channel channel, ChannelBuffer buffer, Request req) throws IOException {  
    Serialization serialization = getSerialization(channel, req);  
    // header.  
    byte[] header = new byte[HEADER_LENGTH];  
    // set magic number.
```

```
    Bytes.short2bytes(MAGIC, header);  
    // set request and serialization flag.  
    header[2] = (byte) (FLAG_REQUEST | serialization.getContentTypeId());  
    if (req.isTwoWay()) {  
        header[2] |= FLAG_TWOWAY;  
    }  
    if (req.isEvent()) {...}  
    // set request id.  
    Bytes.long2bytes(req.getId(), header, off: 4);  
    // encode request data.  
    int savedWriteIndex = buffer.writerIndex();  
    buffer.writerIndex(savedWriteIndex + HEADER_LENGTH);  
    ChannelBufferOutputStream bos = new ChannelBufferOutputStream(buffer);  
  
    if (req.isHeartbeat()) {...} else {  
        ObjectOutput out = serialization.serialize(channel.getId(), bos);  
        if (req.isEvent()) {...} else {  
            encodeRequestData(channel, out, req.getData(), req.getVersion());  
        }  
        out.flush();  
    }
```

Invocation



客户端服务调用Invoke流程:

客户端异步调用实现:

```
private CompletableFuture<Channel channel, Request request, int timeout> {  
    this.channel = channel;  
    this.request = request;  
    this.id = request.getId();  
    this.timeout = timeout > 0 ? timeout : channel.getUrl().getPositiveParameter(TIMEOUT_KEY, DEFAULT_TIMEOUT);  
    // put into waiting map.  
    FUTURES.put(id, this);  
    CHANNELS.put(id, channel);  
}  
  
private static final Map<Long, CompletableFuture< FUTURES = new ConcurrentHashMap<>();
```


客户端服务调用Invoke流程:

客户端拿到结果:

```
private void doReceived(Response res) {  
    if (res == null) {  
        throw new IllegalStateException("response cannot be null");  
    }  
    if (res.getStatus() == Response.OK) {  
        this.complete(res.getResult());  
    } else if (res.getStatus() == Response.CLIENT_TIMEOUT || res.getStatus() == Response.SERVER_TIMEOUT) {  
        this.completeExceptionally(new TimeoutException(res.getStatus() == Response.SERVER_TIMEOUT, channel, res.getErrorMessage()));  
    } else {  
        this.completeExceptionally(new RemotingException(channel, res.getErrorMessage()));  
    }  
}
```


服务端服务调用Invoke流程:

```
pipeline.addLast( name: "decoder", adapter.getDecoder());  
pipeline.addLast( name: "encoder", adapter.getEncoder());  
pipeline.addLast( name: "handler", nettyHandler);
```

decoder, 请求解析, nettyHandler处理, 最后触发到RequestHandler

服务端服务调用Invoke流程:

```

Invoker<?> getInvoker(Channel channel, Invocation inv) throws RemotingException {
    boolean isCallBackServiceInvoke = false;
    boolean isStubServiceInvoke = false;
    int port = channel.getLocalAddress().getPort();
    String path = (String) inv.getObjectAttachments().get(PATH_KEY);

    //if it's stub service on client side(after enable stubevent, usually is set up onconnect or ondisconnect method)
    isStubServiceInvoke = Boolean.TRUE.toString().equals(inv.getObjectAttachments().get(STUB_EVENT_KEY));
    if (isStubServiceInvoke) {
        //when a stub service export to local, it usually can't be exposed to port
        port = 0;
    }

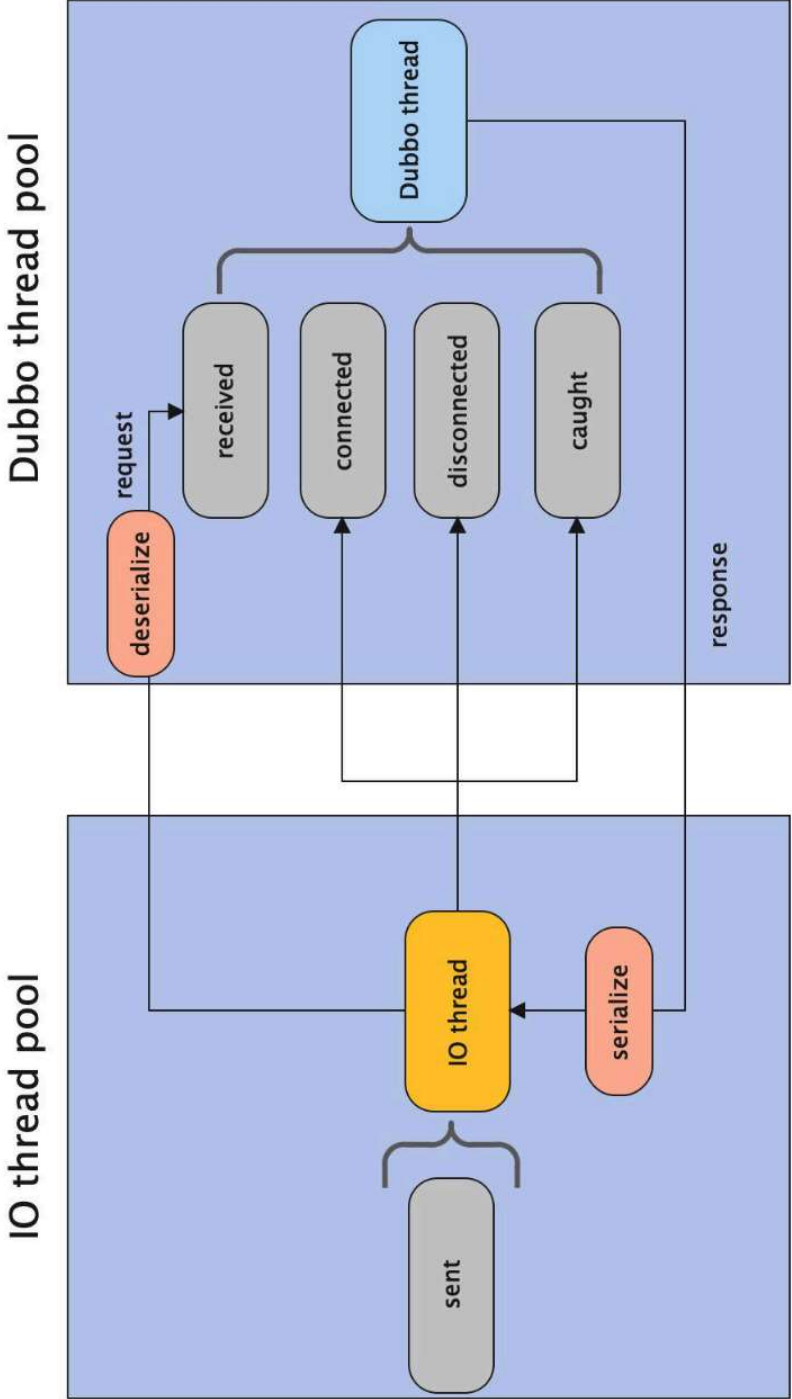
    // if it's callback service on client side
    isCallBackServiceInvoke = isClientSide(channel) && !isStubServiceInvoke;
    if (isCallBackServiceInvoke) {...}

    String serviceKey = serviceKey(
        port,
        path,
        (String) inv.getObjectAttachments().get(VERSION_KEY),
        (String) inv.getObjectAttachments().get(GROUP_KEY)
    );
    DubboExporter<?> exporter = (DubboExporter<?>) exporterMap.getExport(serviceKey);

```

服务端服务调用Invoke流程:

dubbo服务端线程模型，也是扩展点，默认All Dispatcher
ThreadPool默认ThreadNum=200, queues=0



分享与交流

1. dubbo服务端暴露流程
2. dubbo客户端引用流程
3. cDubbo的改动
4. 目前版本2.7, dubbo 3.0 云原生
5. 未来的RPC?

